PLOS One

RESEARCH ARTICLE

Near-optimal P300 speller performance using large language models: A multi-model analysis with performance bounds

Nithin Parthasarathy[1], James Soetedjo[2], Saarang Panchavati[3], Nitya Parthasarathy[4,5], Dongwoo Lee[3], Corey Arnold[3], Nader Pouratian[6], William Speier[3]*

1 Department of Computer Science, University of Illinois at Urbana-Champaign, Urbana, Illinois, United States of America, **2** Department of Bioengineering, University of Washington, Seattle, Washington, United States of America, **3** Department of Radiological Sciences, University of California, Los Angeles, California, United States of America, **4** PE Investments, Boston, Massachusetts, United States of America, **5** Department of Computer Science, Massachusetts Institute of Technology, Cambridge, Massachusetts, United States of America, **6** Department of Neurological Surgery, University of Texas Southwestern Medical Center, Dallas, Texas, United States of America

* speier@ucla.edu

Check for updates

OPEN ACCESS

Citation: Parthasarathy N, Soetedjo J, Panchavati S, Parthasarathy N, Lee D, Arnold C, et al. (2026) Near-optimal P300 speller performance using large language models: A multi-model analysis with performance bounds. PLoS One 21(10): e0349281. https://doi.org/10.1371/journal.pone.0349281

Editor: Onder Aydemir, Karadeniz Technical University: Karadeniz Teknik Universitesi, TÜRKIYE

Received: May 6, 2026

Accepted: August 24, 2026

Published: October 1, 2026

Copyright: © 2026 Parthasarathy et al. This is an open access article distributed under the terms of the Creative Commons Attribution License, which permits unrestricted use, distribution, and reproduction in any medium, provided the original author and source are credited.

Data availability statement: Dataset/Code website and availability has been provided in the paper.

Abstract

Amyotrophic lateral sclerosis (ALS), a progressive neurodegenerative disease, severely impairs communication, requiring assistive technologies that restore interaction. The P300 speller brain-computer interface (BCI) enables communication by translating EEG responses into text; however, its practical adoption is limited by slow typing speed and the need for subject-specific calibration. Recent work has demonstrated that large language models (LLMs), such as GPT-2, can significantly improve the performance of the P300 speller by predicting words. However, it remains unclear whether these gains are model-specific or represent a broader trend across language models. Furthermore, the framework-specific performance bounds of LLM-assisted P300 spellers have not been systematically characterized within a unified decoding framework. In this study, we address these gaps through a systematic multi-model theoretical analysis framework. We evaluate a wide range of language models and introduce an idealized LLM to establish upper bounds on achievable performance. In addition, we incorporate cross-subject classifier training to reduce calibration requirements and assess generalization across subjects. Using extensive simulations on EEG data from 78 subjects, we demonstrate that the evaluated models consistently achieve substantial improvements in typing speed, with gains of up to ∼45% (within-subject training) and ∼75% (across-subject training) over conventional approaches. More importantly, we show that several high-performing models, despite architectural differences, operate within 5% of the theoretical performance bound, indicating diminishing returns from further model scaling. These improvements generalize across both within-subject and across-subject classifiers. Our results suggest that LLM-assisted P300 spellers are approaching oracle-based upper bounds within

Funding: The author(s) received no specific funding for this work.

Competing interests: The authors have declared that no competing interests exist.

the considered decoding framework, shifting the primary bottleneck from language modeling to neural signal decoding. This work provides both a practical framework for improving BCI communication and a theoretical perspective on its achievable limits.

Introduction

Amyotrophic lateral sclerosis (ALS) is a progressive neurodegenerative disease that severely impairs communication [1]. Brain-computer interfaces (BCIs) [2]- [3], and in particular the P300 speller, provide a non-invasive means of restoring communication by translating neural responses into text. In the P300 speller, a character matrix—referred to here as a *flashboard*—is presented to the subject, and the resulting EEG response to highlighted characters is recorded, processed, and interpreted [3,4]. Despite its clinical promise, practical deployment of P300 spellers remains limited by low communication speed and the need for subject-specific calibration [5].

A promising direction to address these limitations is the integration of natural language processing (NLP) into the P300 decoding pipeline. Early work from our group and others demonstrated that character-level and n-gram language models can improve speller performance by exploiting the statistical structure of natural language [6–10]. However, these approaches treat character selections as largely independent events and provide limited gains from context beyond a few preceding characters. More recently, we demonstrated that large language models (LLMs)—specifically GPT-2—can substantially improve P300 speller performance through word-level prediction, and that these gains generalize across subjects via cross-subject classifier training [11,12]. That study established GPT-2 as a strong baseline and introduced the cross-subject evaluation framework (ASCV) used here. However, two important questions were left unanswered: (i) whether these gains are specific to GPT-2 or represent a general property of language models broadly, and (ii) how closely practical systems approach oracle-based upper bounds within this decoding framework. The current study addresses these gaps directly.

Recent articles have further demonstrated the growing interest in the spelling of P300 assisted by LLM. Lebedev et al. [13] used existing offline P300 speller data to investigate whether an LLM could improve text output by compensating for errors introduced by imperfect BCI decoding. Hong et al. [14] introduced ChatBCI, an online P300 speller system using context-driven word prediction with GPT-3.5. These studies provide important evidence that LLMs can improve P300 speller performance in both offline and online settings. However, they do not address whether such gains are model-specific or general across language-model architectures, nor do they establish formal performance bounds for LLM-assisted P300 spelling. The present study addresses these complementary questions through a systematic multi-model analysis and theoretical bound framework.

To this end, we evaluate a broad range of open-weight language models spanning encoder- and decoder-based architectures of varying scales, from encoder-based models such as RoBERTa [15] and BERT [16] to decoder-based models such as

GPT-2 [17], LLaMA [18,19], Falcon [20], and others [21–28]. We introduce a formal analysis framework—oracle-based performance bounds and a physiological timing ceiling—to evaluate how closely practical systems approach the best achievable performance under the assumptions of this LLM-assisted P300 speller architecture. Together, these contributions provide a system-level characterization of performance limits that cannot be inferred from single-model studies.

The central objective of this study is to characterize the performance limits of LLM-assisted P300 spellers. To this end, we introduce an idealized LLM capable of perfect word prediction to establish an upper bound on achievable performance. This bound is then used to evaluate how closely these models approach optimal operation. In addition, we investigate the role of cross-subject classifier training—introduced in our prior work [12]—in reducing or potentially eliminating the need for subject-specific calibration, and assess whether performance gains generalize across subjects and model architectures.

As will be shown, several high-performing LLM-based schemes operate within a small margin ($\sim 5\%$) of the oracle-based word bound, indicating limited remaining gain from language-model improvement for those schemes within the evaluated decoding framework. While the experimental framework, dataset, SWLDA classifier, and baseline decoder structure follow our prior GPT-2-based P300 speller study [12], the present work makes several distinct contributions beyond that earlier publication. First, instead of evaluating a single GPT-2-based word-prediction system, we perform a systematic multi-model comparison across a broad set of encoder- and decoder-based language-model architectures. Second, we introduce oracle-based character and word performance bounds that quantify the maximum achievable improvement attributable to language-model priors within this specific decoding framework. Third, we analyze convergence across modern language models and show that several architectures achieve similar performance near the word bound, suggesting diminishing returns from further language-model scaling. Finally, by comparing practical model performance with both oracle bounds and a physiological timing ceiling, we identify neural decoding and signal acquisition, rather than language modeling alone, as the dominant remaining bottleneck. Thus, the present study extends the prior GPT-2 work from a single-model performance demonstration to a broader system-level analysis of language-model-assisted P300 speller limits.

Our technique involves extensive *offline* simulations using multi-subject EEG data, incorporating layered word completion algorithms (e.g., GPT-2, LLaMA) in conjunction with Dijkstra's algorithm. This framework enables systematic comparison of performance across models and classifier configurations, including both *within*-subject and *across*-subject training. Additional system-level optimizations, including flashboard design and scanning strategies, are incorporated to assess their interaction with language-model-based decoding.

The simulation techniques developed in this work allow for the comparison of many speller scenarios on large and representative targets. These include OOV words that would otherwise be infeasible in an online study due to the significant time requirements. Evaluation of these findings in online settings will be the focus of future work.

Contributions

Specifically, this work makes the following key contributions:

- **Extension beyond prior GPT-2 work:** We build on the previously published GPT-2-based P300 speller framework by evaluating whether the observed gains generalize across a broad set of language-model architectures rather than being specific to GPT-2.
- **Multi-model generalization analysis:** We evaluate modern language models across different architectures and scales, demonstrating that performance gains in P300 spellers are largely model-agnostic within the evaluated framework.
- **Convergence of high-performing model performance:** We show that several high-performing LLM-based schemes operate within $3-5\%$ of the theoretical word bound, revealing a previously unreported convergence phenomenon and indicating diminishing returns from further model scaling within this decoding framework.

- **Bottleneck identification:** We demonstrate that the primary limitation in modern P300 spellers has shifted from language modeling to neural signal decoding and physiological constraints.
- **Cross-subject validation:** We show that these results generalize across both within-subject and across-subject training paradigms, supporting the feasibility of reducing or potentially eliminating subject-specific calibration.

A block diagram of the complete P300 speller BCI system is shown in Fig 1. The organization of this paper is as follows. The Materials and methods section describes the techniques and algorithms used for the flashboard, stimuli, classifiers, and character and word prediction. The Results section then presents the performance evaluation, followed by the Discussion and conclusions.

Materials and methods

The methods employed in this study follow the framework established in our prior work [12]. We summarize each component below, noting where the current study introduces new elements or departs from the prior approach.

Ethics statement

This study was conducted according to the principles embodied in the Declaration of Helsinki and according to local statutory requirements. The study was approved by the UCLA Institutional Review Board (IRB #11–002062; initial approval 21 June 2011, continuously active). All participants provided written informed consent prior to participation. Subject recruitment was carried out between 21 June 2011 and 1 January 2021.

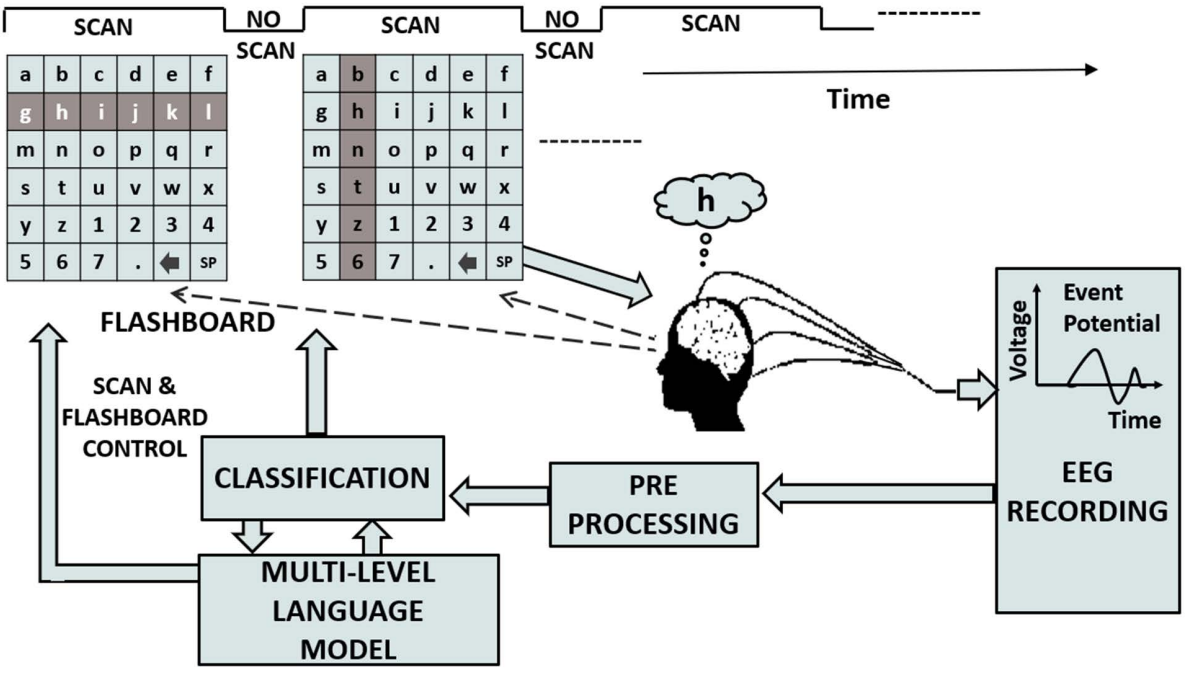


Fig 1. A block diagram of the P300 speller based BCI used in this work. Subject EEG response to character highlights on a flashboard are shown.

https://doi.org/10.1371/journal.pone.0349281.g001

Language models

This section describes the multi-level character language model used for flashboard probability estimation and out-of-vocabulary (OOV) handling, following the approach of [12]. Character frequency in English motivates the scanning order: vowels occur more frequently than consonants [29], and are therefore prioritized earlier in the flashboard sequence. Conversely, less frequent consonants, such as *'x'* and *'z'*, are highlighted later.

Smoothing techniques [30] enable transitions across models when words are encountered outside the vocabulary (OOV), blending higher-order and lower-order models. As illustrated in Fig 2, the language model spans a hierarchy ranging from simple trigram models to more complex biword models.

As an example, consider the word *'cat'* and suppose *'ca'* has already been spelled. If *'cat'* is not present in the language model, the conditional probability $p(\text{'}t\text{'} \mid \text{'}ca\text{'})$ is zero. Smoothing addresses this by using alternative contexts, such as $p(\text{'}t\text{'} \mid \text{'}a\text{'})$, where characters have been observed to co-occur.

Among several smoothing techniques evaluated, Kneser–Ney smoothing was selected for its strong OOV performance [31]. This approach recursively combines higher-order and lower-order n-gram models to produce robust probability estimates.

The frequency counts used in the language model were estimated from the Brown Corpus [32], a widely used benchmark dataset that provides broad coverage of written American English. The complete recursive formulation of the smoothing equations is provided in S1 Appendix.

Data collection

The dataset used here is identical to that described in our prior work [12]; we summarize the key parameters for completeness. Data for offline analyses were obtained from 78 healthy volunteers with normal or corrected-to-normal vision between the ages of 20 and 35. Fig 3 shows the BCI display flashing before a volunteer whose response is being

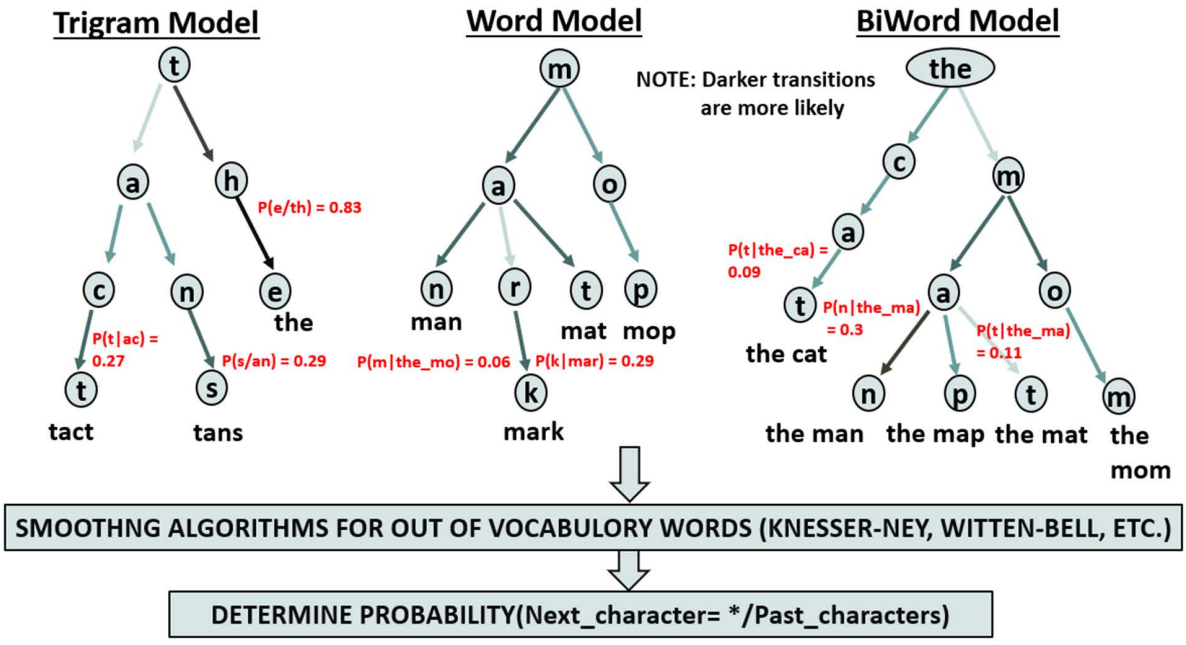


Fig 2. Types of language models used. Starting with the simplest on the left to the most complex on the right.

https://doi.org/10.1371/journal.pone.0349281.g002

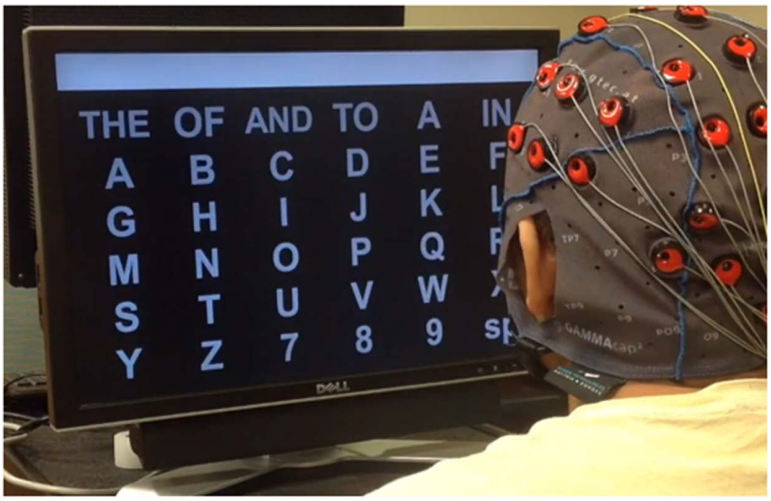


Fig 3. BCI display and volunteer EEG recording during stimulus presentation.

https://doi.org/10.1371/journal.pone.0349281.g003

recorded. All data were acquired using GTEC amplifiers, active EEG electrodes and electrode cap (Guger Technologies, Graz, Austria); sampled at *256 Hz*, referenced to the left ear; grounded to *AF_Z*; and filtered using a bandpass of *0.1–60 Hz*. The set of electrodes consisted of 32 channels placed according to a previously published configuration [33]. We used a window of 600 ms at 256 Hz, resulting in 154 time points per channel. We then downsampled the signal by a factor of 12 to obtain a feature vector that was $32 \times 13 = 416$ features. The system also used a 6×6 character grid, row and column flashes, and a stimulus duration of 100 msec and an interval between stimulus of 25 ms for a stimulus onset asynchrony of 125 msec. The channels used were *Fpz, Fz, FC1, FCz, FC2, FC4, FC6, C4, C6, CP4, CP6, FC3, FC5, C3, C5, CP3, CP5, CP1, P1, Cz, CPz, Pz, POz, CP2, P2, PO7, PO3, O1, Oz, O2, PO4, PO8* again based on a published montage [33].

Feature extraction: training methodology

The stepwise linear discriminant analysis (SWLDA) as described by Speier et al. [8] and originally used in [4] was used as a classifier. SWLDA is a stepwise method that sequentially adds or removes features based on their statistical significance, optimizing the model for better classification. It was chosen for its efficiency in handling high-dimensional data and its proven effectiveness in BCI applications [34].

SWLDA utilizes a discriminant function, which is determined in the training step [35]. The SWLDA classifier was trained using one of two methods: Across-subject cross-validation (ASCV) (also known as leave-one-subject-out cross-validation) and within-subject cross-validation (WSCV). Although both use SWLDA, note that these two techniques fundamentally differ in how entire subject responses are split between what was used for training and what was used for testing.

a) **ASCV:** Of the *N* subjects, one subject was chosen as the test subject, while the data from the other $N-1$ subjects were used to train the SWLDA classifier. The SWLDA classifier then attempted to take the data of the test subject and spell out the target phrase. Note that this technique finally produces one classifier table for all *N* subjects.

b) **WSCV:** In WSCV, a three-fold cross-validation was performed using SWLDA on the EEG data of each individual subject (unlike ASCV), resulting in a unique classifier output for each subject. More specifically, the output phrase contained at least 20 characters and had 120 flashes for each character which was split into three sets. During the first test iteration, the first set was chosen as the test phrase, while the other two sets were used to train the SWLDA classifier.

On completion, the same procedure would repeat, but with the second and third sets being the test phrases during the second and third trial, respectively. This method would then be replicated on all subjects.

During training, class labels were predicted using ordinary least squares. Next, the forward step-wise analysis adds the most important features, and the backward analysis step determines and removes the least important features. This process repeats until the number of features remains constant after a set number of iterations or until the number of required features is met. The final selected features are then added to the discriminant function. The flash score *i* for the character *t*, y_t^i can then be calculated as the dot product of the feature weight vector with the features of the signal from that trial.

Virtual flashboard design using a language model

The virtual flashboard schemes used in this study follow [12]. In brief, the highlighted set of characters is modified without changing the underlying flashboard structure, allowing the approach to overlay onto any existing flashboard in current use. Two schemes are evaluated:

a) **Sequential flashboard:** Characters are highlighted in frequency-weighted order rather than alphabetically, so that higher-frequency characters are flashed earlier. See Fig 4 and reference [12] for details.

b) **Diagonal flashboard:** The most likely characters are arranged along diagonals (Fig 5), forcing contextually similar characters to flash separately. The most likely letters *e, t, a, i, n, o* are placed on the main diagonal, with less likely letters on adjacent diagonals.

Further optimization of the scanning order

Weighted scanning order and dynamic stopping follow [12]. In brief, weighted scanning highlights character sets in order of probability rather than round-robin. Dynamic stopping terminates the current character flash sequence as soon as a decision threshold is reached, immediately beginning the next character. Dynamic stopping was applied in all simulation conditions reported in Table 2 as well as in the physiological ceiling computation in the Theoretical performance bound and Shannon limit section.

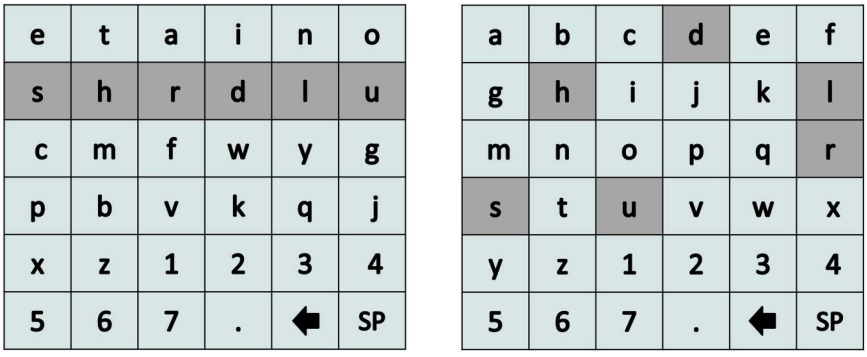


Fig 4. Mapping of probabilistic flashing onto traditional alphabetical flashboard. a) Probabilistic flashboard highlighting order (Prob(Next character/Earlier spelled characters)) in a sequential frequency-sorted flashboard. SP denotes space between words. b) Sequential flashboard characters shown in panel a are virtually mapped onto a conventional alphabetical flashboard.

https://doi.org/10.1371/journal.pone.0349281.g004

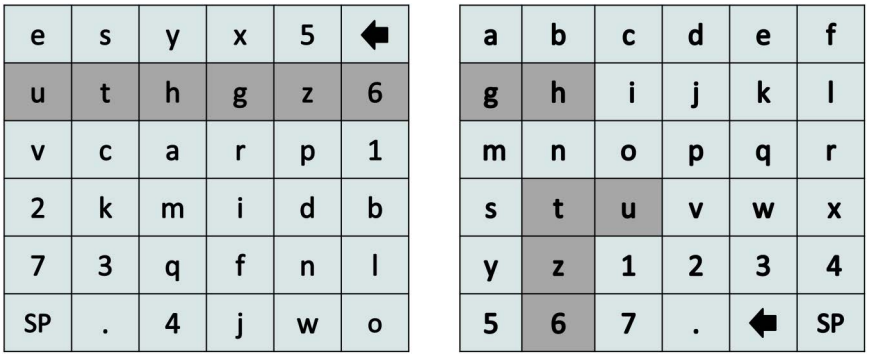


Fig 5. Mapping of the diagonal flashboard onto the traditional alphabetical flashboard. (a) Highlighting in a diagonal flashboard design where the probabilities represent Prob(Next character/Earlier spelled characters). Most likely characters e, t, a, i ... are organized along diagonals. Note that SP denotes space between words. b) Diagonal highlighted flashboard characters in panel a are virtually mapped onto a conventional alphabetical flashboard.

https://doi.org/10.1371/journal.pone.0349281.g005

Word prediction algorithms

Similarly to word completion suggestions that are available when messaging using smartphones, flashboard efficiency can be likewise increased by replacing unlikely characters with word choices. This section describes algorithms to compute these word choices.

a) **Dijkstra's Algorithm:** The classic technique for traversing a tree or a graph is either a depth-first search or a breadth-first search, which are well known algorithms [36]. Although computationally efficient, a key limitation is that it does not provide a technique for evaluating OOV words, which occur when the partially spelled word is absent from the language model. In contrast, our approach is to use Dijkstra's algorithm [37] for word prediction. This is implemented in the form of a trellis similar to dynamic programming techniques as shown in Fig 6. As shown in the figure, every node

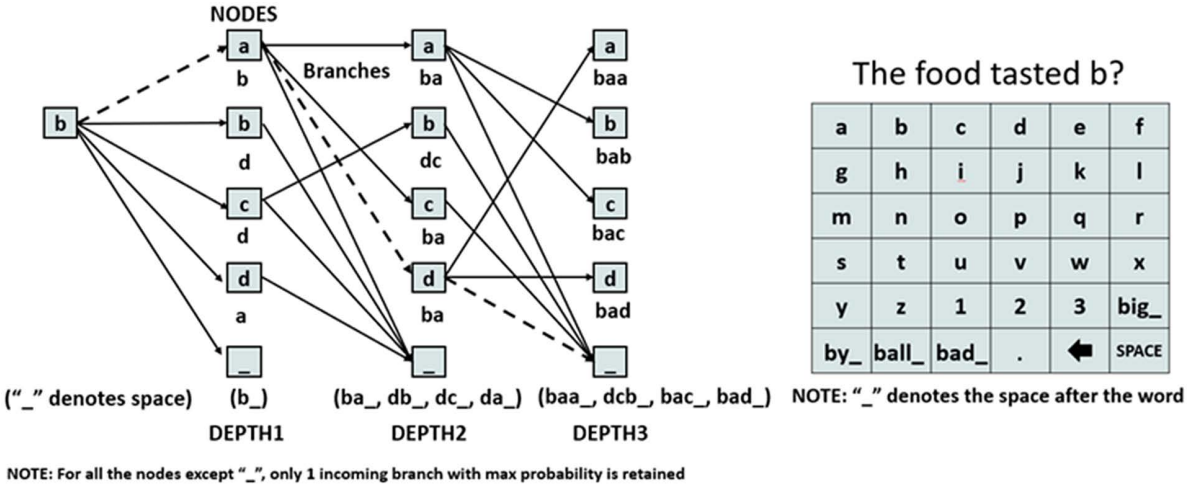


Fig 6. Application of Dijkstra's algorithm to produce word suggestions. (a) Implementation of Dijkstra's algorithm to find most likely word completions. (b) Corresponding flashboard showing final most likely word completions with four characters replaced by the word choices. Note: "_" represents space between words.

https://doi.org/10.1371/journal.pone.0349281.g006

in a trellis stage has *M* incoming possibilities (where *M* is the number of unique characters on the flashboard) out of which only the one with the highest probability is retained. In doing so, the complexity is reduced, as every possibility does not have to be exhaustively evaluated. Further, as more stages are incorporated into the trellis, longer length word choices are obtained, resulting in higher evaluation time and complexity. For each branch at a given stage, probabilities are determined by using the frequency count of branch characters given prior characters using the language model along with smoothing. At every stage, the highest probability completions then provide the most likely word completions.

b) **Large Language Models:** Numerous open-weight language models were used in this evaluation, including RoBERTa [15], BERT [16], XLNet [22], BART [21], OPT [25], GPT-J [26], GPT-Neo [23], and GPT-2 [17,38]. This set was complemented by more recent open-weight models including GPT-NeoX [27], BLOOM [24], Falcon [20], MPT [28], LLaMA 3.1 [18], and LLaMA 3.3 [19]. These models span encoder-based, decoder-based, sequence-to-sequence, and autoregressive architectures, enabling evaluation of whether language-model-assisted P300 speller gains are specific to GPT-2 or generalize across model families. Although GPT-4 and GPT-5 have since been released, they are not open-source and do not readily expose word-prediction probabilities through the same reproducible interface, precluding their use here.

Language-model inference and word-suggestion generation

For each decoding step, the language model was provided with the text already decoded up to the current cursor position as context. The goal of the language model was to generate candidate next-word suggestions that could be displayed on the virtual flashboard. A maximum of six word suggestions was used, consistent with the number of replacement positions available in the flashboard.

Because the evaluated models use different tokenization schemes and output formats, model outputs were post-processed into complete word suggestions before being displayed. For models using subword tokenization, generated tokens were converted into whole-word candidates by retaining only valid completed word suggestions. Duplicate, incomplete, malformed, or non-alphabetic candidates were removed. The resulting candidate list was then mapped to the available word-suggestion positions on the flashboard.

For decoder-based models such as GPT-2, GPT-J, GPT-Neo, GPT-NeoX, OPT, BLOOM, Falcon, MPT, and LLaMA, the preceding decoded text was used as the prompt context for next-word prediction. For encoder-based or masked-token models such as BERT and RoBERTa, word suggestions were generated using a masked-token formulation at the current word position. For sequence-to-sequence or permutation-based models such as BART and XLNet, candidate generation was similarly constrained to the next-word prediction task using the available decoded context.

The raw outputs of different language models were not interpreted as directly comparable calibrated probabilities across architectures. Instead, each model was evaluated within the same downstream P300 decoding and flashboard framework. Candidate suggestions produced by each model were mapped to the same display positions and then combined with the P300 decoder evidence through the decoding procedure described below. This ensured that differences in performance reflected the usefulness of the generated word suggestions within the speller framework rather than differences in model-specific probability calibration.

If fewer than six valid word suggestions were produced by a language model, the remaining suggestion slots were filled using the Dijkstra-based word-completion method (Fig 7). This fallback mechanism also handled out-of-vocabulary cases and ensured that every decoding step had a complete set of candidate character or word choices. Thus, modern language models were used as a higher-level word-prediction layer, while Dijkstra-based completion provided a robust fallback for incomplete, invalid, or out-of-vocabulary predictions.

Inference latency, memory usage, and computational cost were not formally benchmarked in this study, as it was conducted entirely through offline simulation; the reported ITR values therefore reflect decoding performance under the

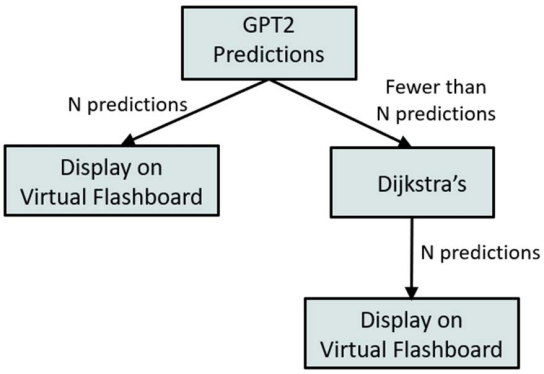


Fig 7. Layered GPT-2 word choice selection to obtain most likely N word completions with fallback to Dijkstra's algorithm.

https://doi.org/10.1371/journal.pone.0349281.g007

evaluated language-model-assisted word-suggestion framework rather than full end-to-end real-time system latency. These costs were not expected to be comparable across the evaluated architectures: the models span more than two orders of magnitude in parameter count, from GPT-2 (124M–1.5B parameters) to substantially larger models such as GPT-J (6B), GPT-NeoX (20B), MPT (30B), and LLaMA 3.1/3.3 (8B–70B), each requiring correspondingly greater memory and compute. Despite this disparity in computational footprint, the best-performing LLM-based schemes converged within a narrow performance range, reinforcing the observation that further increases in model scale and inference cost are unlikely to yield proportional gains in P300 speller performance within this decoding framework. Real-time deployment would require additional measurement and optimization of inference delay, particularly for the larger models evaluated here.

Performance bounds

To quantify how closely practical systems approach optimal performance, we define two theoretical performance bounds: a character bound and a word bound. Each bound is constructed by replacing the language-model prior with an oracle prior that assigns maximum probability to the correct prediction at every decoding step. These bounds are defined within the same decoding framework described in the Decoder section, so the only difference between a practical system and its corresponding bound is the accuracy of the prior. Note that these bounds should not be interpreted as general information-theoretic capacity limits of P300 communication. Instead, they are framework-specific oracle bounds defined by the assumptions of the present system, including the flashboard design, candidate word set, dynamic stopping rule, out-of-vocabulary fallback mechanism, correction procedure, and SWLDA-based neural decoder. Their purpose is to isolate the maximum performance gain attributable to language-model priors within this controlled framework.

Character bound. At each decoding step, the oracle assigns a prior probability of 1 to the correct next character and 0 to all other characters. Formally, let x^* denote the correct next character. The oracle prior is then

$$P_{\text{oracle}}(x_t \mid x_{t-1}, \ldots, x_0) = \begin{cases} 1 & \text{if } x_t = x^* \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

This prior is substituted directly into the posterior in Equations 4–5 in place of the language model prior. Because the prior concentrates all probability mass on the correct character, the decoder requires fewer flash sequences to accumulate sufficient evidence to cross the decision threshold P_{Thresh}, thus minimizing the selection time. The resulting ITR constitutes an upper bound on what any character-level language model can achieve within this decoding framework, regardless of model architecture or complexity.

Word bound. The word bound is defined analogously at the word level. At each word boundary, the oracle assigns a prior probability of 1 to the correct next complete word among the $K \leq 6$ word suggestions displayed on the flashboard, and zero probability to all other word candidates:

$$P_{\text{oracle}}(w_t \mid w_{t-1}, \ldots, w_0) = \begin{cases} 1 & \text{if } w_t = w^* \\ 0 & \text{otherwise} \end{cases} \tag{2}$$

When the correct word appears among the K suggestions, selection requires only a single confirmation flash sequence, yielding the minimum achievable selection time for word-level decoding. When the correct word is absent from the suggestion set—as occurs with out-of-vocabulary (OOV) words—the system falls back to character-level decoding using the character oracle prior of Equation (1), ensuring the bound remains well-defined across all tokens in the test corpus.

Validity as upper bounds. These bounds are valid upper bounds for the following reason: the oracle prior eliminates all uncertainty attributable to the language model while leaving the neural decoding process, governed by the EEG signal quality and the SWLDA classifier, unchanged. Any practical language model can at best approximate this oracle, so no real system operating within this framework can exceed the corresponding bound. Crucially, the bounds are subject-specific: because ITR depends on individual P300 signal strength, captured in the attended and non-attended distributions $f(y_t^j \mid x_t)$ of Equation (3), the bound for each subject reflects that subject's neural signal fidelity. The bound values reported in Table 2 represent averages across all 78 subjects, consistent with how practical scheme ITRs are reported throughout. It is important to note that these bounds are defined within the constraints of the decoding framework used in this study and do not represent information-theoretic capacity limits of the P300 communication channel. Rather, they quantify the maximum achievable performance under ideal language-model priors while preserving the same neural decoding assumptions. As such, they should be interpreted as practical upper bounds for systems operating within this architecture.

What the bounds capture and do not capture. The character and word bounds isolate the contribution of the language model to overall system performance. The gap between a practical scheme and its corresponding bound reflects residual language modeling error: that is, cases in which the model fails to assign sufficiently high probability to the correct prediction. The gap between the word bound and the physiological ceiling derived in the Theoretical performance bound and Shannon limit section reflects limitations in neural signal decoding that no language model improvement can address. Together, these two bounds partition the total system performance gap into a language modeling component and a neural decoding component, providing a principled basis for identifying where future research effort is most likely to yield meaningful returns.

Decoder

In this paper, a simple decoding scheme based on thresholding is used to deduce the character or word. First, the score for each flash is computed by $y_t^j = \mathbf{w} \cdot \mathbf{z}_t^j$ where the weight vector of the feature is determined by extracting the feature as described in the previous subsection. With the assumption that the distributions were Gaussian [8], the *'attended'* and *'non-attended'* signals (note that this terminology refers to highlighted and non-highlighted characters on the flashboard) were found and given by the authors.

$$f(y_t^j \mid x_t) = \begin{cases} \frac{1}{\sqrt{2\pi\sigma_a^2}} e^{-\frac{(y_t^j - \mu_a)^2}{2\sigma_a^2}} & x_t \in A_i^t \\ \frac{1}{\sqrt{2\pi\sigma_n^2}} e^{-\frac{(y_t^j - \mu_n)^2}{2\sigma_n^2}} & x_t \notin A_i^t \end{cases} \tag{3}$$

where $\mathbf{A}_t^i$ is the set of characters illuminated for the ith flash for the character t in the sequence. $\mu_a, \sigma_a^2, \mu_n, \sigma_n^2$ are the means and variances of the distributions for the *att*ended and non-attended flashes, respectively.

$$P(x_t \mid \mathbf{y}_t, x_{t-1}, ..., x_0) = \frac{P(x_t \mid x_{t-1}, ..., x_0)\, P(\mathbf{y}_t \mid x_t)}{P(\mathbf{y}_t \mid x_{t-1}, ..., x_0)} \tag{4}$$

$$P(x_t \mid \mathbf{y}_t, x_{t-1}, ..., x_0) = \frac{1}{Z} P(x_t \mid x_{t-1}, ..., x_0) \prod_i f(y_t^i \mid x_t) \tag{5}$$

where $P(x_t|x_{t-1}, ..., x_0)$ is the prior probability of a character given the history, $f(y_t^i|x_t)$ are the probability density functions from equation 3, and Z is a normalizing constant. The language model is used to derive the prior as per

$$P(x_t \mid x_{t-1}, ..., x_0) = Count(x_t, x_{t-1}, ..., x_0) / \sum_{x_t} Count(x_t, x_{t-1}, ..., x_0) \tag{6}$$

where $Count(x_t, x_{t-1}, ..., x_0)$ is the number of occurrences of the string $x_t x_{t-1} ... x_0$ in the corpus. The threshold probability, P_{Thresh}, is then set to determine when a decision should be made. In all our simulations, P_{Thresh} was set to 0.95, consistent with the value used in prior work [11,12]. The program flashes characters until $\max_{x_t} P(x_t \mid \mathbf{y}_t, x_{t-1}, ..., x_0) \geq P_{\text{Thresh}}$ or the number of flash sets reaches a predetermined maximum value. The classifier then selects the character that satisfies $\arg\max_{x_t} P(x_t|\mathbf{y}_t, x_{t-1}, ..., x_0)$.

The subject can correct typing errors by invoking the backspace option provided in the flashboard. In the simulations reported here, errors are corrected during decoding, so the final output text contains no residual errors after correction. This zero-residual-error condition should therefore be interpreted as a corrected-output assumption rather than as evidence that no errors occur during spelling. The correction process can impose additional user burden because an incorrect character or word selection must first be deleted and then re-entered.

Accordingly, the reported ITR values include the time required for simulated correction steps when errors occur, and should be interpreted as corrected-output performance rather than as evidence that no incorrect selections occurred during spelling.

To characterize correction burden directly, we extracted pre-correction accuracy, correction rate, and flash overhead from the available archived simulation logs for the retained model and bound configurations (Table 1). Pre-correction accuracy—estimated as $N_{\text{chars}}/(N_{\text{chars}} + N_{\text{corr}})$, where $N_{\text{chars}} = 7{,}892$ and N_{corr} is the number of backspace correction events—had a median of at least 99% across the retained configurations. The median correction rate ranged from 0.11% to 0.94%, and the median overhead from correction-related flash sequences ranged from 1.0% to 3.4% of total stimulus presentations. These results indicate that the zero-residual-error outputs were not achieved through a large median correction burden. However, the ASCV correction-burden summaries exclude seven subjects per condition whose simulations exceeded the 75-scan limit and were assigned ITR = 0, because the corresponding logs were not retained. Thus, the ASCV correction-burden values should be interpreted as conditional on the retained logs, while the ITR values in Table 2 remain the primary performance metric because they include correction time and zero-ITR failures. The analysis of residual error under alternative decision thresholds remains a direction for future work.

Simulation overview

Across all subjects in the dataset, the average waveforms found for the attended and non-attended stimulus responses are shown in Fig 8. Using the EEG signal for the online sessions, the stimulus responses were grouped according to whether they corresponded to the character the subject was currently trying to spell. For each subject, the average

Table 1. Correction-burden metrics derived from archived simulation logs. Pre-correction accuracy is estimated as $N_{chars}/(N_{chars} + N_{corr})$, where N_{chars} = 7,892 (Declaration of Independence) and N_{corr} is the total number of backspace correction events per session. Correction rate is $N_{corr}/N_{chars} \times 100$. Overhead is the percentage of total flash sequences attributable to backspace and re-entry steps. All metrics are medians; distributions are right-skewed owing to a small number of subjects with weak P300 signals. For ASCV, N = 71 because seven subjects per condition exceeded the 75-scan limit and were assigned ITR = 0; their logs are absent from the archived outputs. The WSCV log for BERT and logs for RoBERTa, OPT, GPT-2, and Dijkstra were not retained in the archived outputs.

Model	Mode	N	Pre-corr. acc. (%) Median	Corr. rate (%) Median	Overhead (%) Median
XLNet	WSCV	78	99.07	0.94	2.7
XLNet	ASCV	71	99.73	0.27	1.0
BERT	ASCV	71	99.67	0.33	1.0
BART	WSCV	78	99.53	0.47	2.8
BART	ASCV	71	99.79	0.22	1.1
BLOOM	WSCV	78	99.64	0.36	2.3
BLOOM	ASCV	71	99.77	0.23	1.4
GPT-Neo	WSCV	78	99.68	0.33	2.4
GPT-Neo	ASCV	71	99.81	0.19	1.2
GPT-J	WSCV	78	99.67	0.33	2.2
GPT-J	ASCV	71	99.85	0.15	1.1
GPT-NeoX	WSCV	78	99.70	0.30	2.0
GPT-NeoX	ASCV	71	99.89	0.11	1.0
Falcon7	WSCV	78	99.68	0.33	2.3
Falcon7	ASCV	71	99.86	0.14	1.1
MPT-30b	WSCV	78	99.68	0.33	2.3
MPT-30b	ASCV	71	99.84	0.16	1.3
LLaMA 3.1	WSCV	78	99.72	0.28	2.2
LLaMA 3.1	ASCV	71	99.86	0.14	1.0
LLaMA 3.3	WSCV	78	99.70	0.30	2.3
LLaMA 3.3	ASCV	71	99.81	0.19	1.3
Word Bound	WSCV	78	99.69	0.31	3.4
Word Bound	ASCV	71	99.86	0.14	1.1

https://doi.org/10.1371/journal.pone.0349281.t001

response for attended and non-attended stimuli was produced. Then a global average was produced by averaging these signals across subjects. For this discussion, we focus on four channels that we previously determined to be the most influential in our P300 analyses [39]. In each of these channels, a defined negative peak was observed around 200 ms after attended stimuli, followed by a positive peak around 300 ms. The peaks in the Cz channel were slightly later than the peaks in the parieto-occipital channels. In each of these channels, the difference between the attended and non-attended signals was significant in each of these peaks. In addition, there were smaller peaks in both attended and non-attended signals with an interval of 125 ms, which reflects the stimulus onset asynchrony. These peaks likely reflect attended stimuli before or after the current stimulus due to the overlapping response intervals. Statistical variation from such small data sets would limit the confidence in the simulation results. As a result, a large and meaningful data set is used as simulation data. The text of *the Declaration of Independence* (DoI) is used as simulation data [40]. The full length of this document is 7,892 characters after removing all punctuation and special characters. The DoI was chosen as the simulation dataset as it is concise, well-known, and contains words of varying length, including a number of out-of-vocabulary (OOV) words that stress the simulation. Furthermore, to keep the simulation accurate, numerous subject responses are used to provide a wide range of brain activity.

Table 2. Information transfer rate (mean ± standard deviation format) in bits/min of various schemes for WSCV and ASCV subject training. The scheme with the highest mean ITR is shown in bold.

Scheme	WSCV ITR (bits/min)	ASCV ITR (bits/min)
Random	51.93 ± 11.2	33.95 ± 14.4
Deterministic	53.42 ± 11.4	34.49 ± 14.9
FreqSorted	53.44 ± 10.8	35.44 ± 15.0
Diagonal	54.43 ± 11.0	36.51 ± 15.4
Char Bound	55.86 ± 10.5	37.97 ± 14.6
Dijkstra	63.59 ± 10.1	46.05 ± 17.8
Roberta	59.01 ± 10.6	41.19 ± 16.6
Bert	65.51 ± 10.1	45.25 ± 17.6
XLnet	62.81 ± 9.8	48.18 ± 18.4
Bart	71.64 ± 8.6	55.23 ± 20.1
Bloom	72.49 ± 9.3	56.69 ± 19.7
Opt	75.04 ± 7.4	59.97 ± 20.4
Falcon7	74.91 ± 7.3	59.89 ± 20.3
MPT_30b	74.42 ± 7.4	59.69 ± 20.3
GPT-NeoX_20b	74.72 ± 7.4	59.7 ± 20.3
GPT-Neo	74.92 ± 7.4	**60.22 ± 20.4**
GPT-J	**75.36 ± 7.2**	59.90 ± 20.3
GPT-2	75.31 ± 7.5	59.75 ± 21.0
LLaMA 3.1	75.09 ± 7.2	60.18 ± 20.4
LLaMA 3.3	75.10 ± 7.2	60.17 ± 20.4
Word Bound	78.21 ± 6.4	62.84 ± 21.8

https://doi.org/10.1371/journal.pone.0349281.t002

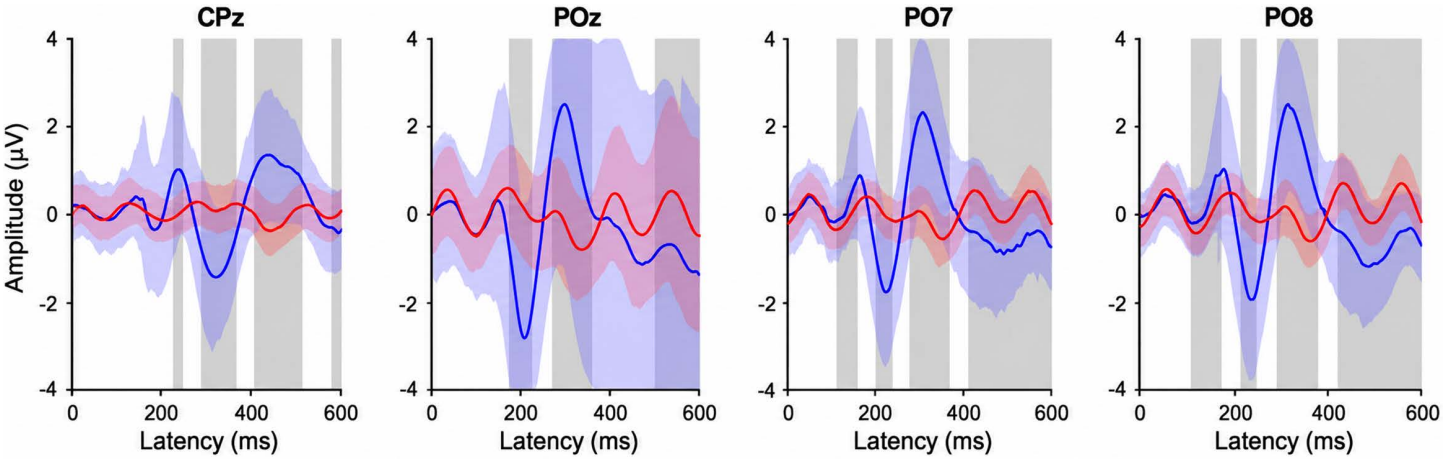


Fig 8. Waveform analysis across all subjects in the dataset for four channels. Average ERPs for target (blue) and non-target (red) stimuli are shown. Shaded regions represent standard error of the mean; gray regions represent significant differences after correction for false discovery rate.

https://doi.org/10.1371/journal.pone.0349281.g008

During simulations with the DoI dataset, the attended and non-attended (once again, this terminology refers to whether the desired character is present or absent in the highlighted flashboard character) scores for a given subject were chosen from the laboratory results for that particular subject. The laboratory results are sampled and subsequently modeled using a two-state Markov process representing attended and non-attended responses. This formulation captures the temporal dependence between successive stimulus responses, where the probability of a given state depends on the immediately preceding state, while preserving the binary structure inherent in P300 detection. This approximation provides a tractable way to model correlations across sequential stimuli without introducing significant additional complexity. Higher-order models were not considered to avoid over-parameterization given the temporal resolution of the EEG signals.

For the two-state Markov model, there are four possible sampling combinations depending on whether the current and previous state was attended or non-attended. When a character or word is decoded, it is compared to the intended character or word. In the event of an error, a *Backspace* is initiated as the subsequent character. Following this, the deleted character/word is flashed again. The net effect of this is to *undo* the errors and as errors are corrected during decoding, the final output text contains no residual errors. However, note that this could possibly require multiple attempts and a long follow-up time. The follow-up time could be significantly restricted in addition to tolerating residual error, and this evaluation could be a potential future study.

Performance evaluation metrics

Information Transfer Rate (ITR) is a crucial metric in the evaluation of Brain-Computer Interfaces (BCIs), quantifying the speed and precision of communication [41]. The metric is attractive for several reasons: It is derived from the principles of information theory, it combines competing statistics of speed and accuracy, and it reduces performance to an information transfer problem that can be compared between applications [42]. Following McFarland et al. [41], it is calculated as:

$$\text{ITR} = \frac{B}{T} \times 60 \text{ bits/min} \tag{7}$$

where B is the information per selection in bits, defined as:

$$B = \log_2(N) + P\log_2(P) + (1-P)\log_2\left(\frac{1-P}{N-1}\right) \tag{8}$$

where N is the number of characters in the speller (grid size), P is the probability of a correct selection, and T is the average time per selection, including all pauses and correction overhead. At perfect accuracy ($P=1$), B reduces to $\log_2(N)$.

It is possible for the system to generate errors faster than the user can correct them. However, in our analysis, in almost all cases, we see high enough accuracies that users can make all corrections as they happen, resulting in a perfect output. This represents an upper-bound evaluation scenario. In practical deployments, systems may allow a small residual error rate in exchange for improved communication speed, resulting in a different operating point on the speed-accuracy trade-off. To allow for finite duration simulation of a particular character or word, when the simulation requires more than 75 flashboard scans, it is abandoned, and an ITR of 0 (100% error rate) is assigned for that particular character or word and the simulation then moves on to the next character or word.

Publications in this area [43,44] have used sensitivity and specificity as performance metrics. However, metrics such as sensitivity and specificity measure the frequency of correct letters in a subject's output. In the current study, all errors are manually corrected, so no errors remain in the output text. Therefore, metrics such as sensitivity and specificity are 100% for all subjects by necessity.

Some errors (e.g., obvious typos) do not affect the readability of text and could therefore be allowed to remain in the final output without affecting the meaning. Allowing these errors to remain could then potentially result in a more efficient

system because it would not waste time forcing the user to make these corrections. Language models could also be used to automatically correct some of these errors [8,45] if they are not manually corrected.

Theoretical performance bound and Shannon limit

To understand the significance of this convergence, it is instructive to place the word performance bound within a broader information-theoretic context. This allows the remaining performance gap to be attributed to specific system components and provides a principled basis for identifying where future improvements are most likely to be productive.

Information content per selection. The general ITR formula defined in Equations (7)–(8) reduces, at perfect accuracy ($P = 1$), to:

$$B = \log_2(N) = \log_2(36) \approx 5.170 \text{ bits/selection.} \tag{9}$$

This quantity is fixed by the grid size and is largely independent of the language model or decoding strategy used.

Physiological minimum selection time. The minimum selection time is constrained by P300 response physiology. With an SOA of 125 ms and 12 flash sets per sequence, the selection time assuming a single flash sequence with dynamic stopping is:

$$T_{\min} = 12 \times 0.125 = 1.5 \text{ seconds/selection.} \tag{10}$$

This represents an optimistic lower bound on selection time, assuming reliable P300 detection from a single flash sequence. In practice, multiple sequences are typically required for reliable detection. A more physiologically conservative estimate, assuming a minimum of two flash sequences, yields:

$$T_{\min}^{(\text{conservative})} = 2 \times 12 \times 0.125 = 3.0 \text{ seconds/selection.} \tag{11}$$

Maximum theoretical ITR. Substituting into the ITR formula gives:

$$\text{ITR}_{\max} = \frac{5.170}{1.5} \times 60 \approx 207 \text{ bits/min (single-sequence bound).} \tag{12}$$

The single-sequence estimate of 207 bits/min represents an idealized upper bound under single-sequence detection assumptions, as reliable P300 detection from a single flash sequence is not typically achievable in practice. Under the conservative two-sequence assumption:

$$\text{ITR}_{\max}^{(\text{conservative})} = \frac{5.170}{3.0} \times 60 \approx 103 \text{ bits/min.} \tag{13}$$

The achievable physiological ceiling therefore lies in the range of approximately $103 - 207$ bits/min.

Results

This section details the performance evaluation of various schemes.

Both the Shapiro–Wilk and Kolmogorov–Smirnov tests [46] reject normality ($p < 10^{-3}$). Because the same subjects were evaluated across decoding schemes, we used nonparametric repeated-measures testing. Specifically, we used the Friedman test as the omnibus test for differences across decoding schemes, followed by selected pairwise Wilcoxon signed-rank [47] comparisons for the selected contrasts of interest, with Holm–Bonferroni correction for multiple comparisons. The Friedman test showed a significant effect of decoding scheme for both within-subject cross-validation (WSCV;

$\chi^2(20) = 1527.74$, $p < 0.001$, Kendall's $W = 0.979$) and across-subject cross-validation (ASCV; $\chi^2(20) = 1377.73$, $p < 0.001$, Kendall's $W = 0.883$). Selected pairwise comparisons are reported in Table 3.

Fig 9 shows the performance of different schemes for the 78 subjects in a *violin* plot along with an embedded traditional box and whiskers plot. The violin plot depicts the estimated probability density of ITR values across subjects using a kernel density estimate, while the embedded box and whiskers plot provides relevant statistics. In this case, the classifier was individually trained on each subject (*WSCV* training of the Feature extraction: training methodology section). A maximum of six word suggestions were provided, with numbers in the flashboard replaced by word suggestions. Note that the final error rate in the output text is zero in all these cases. To clarify, the ITR was calculated as defined in the Performance evaluation metrics section.

Table 2 shows that GPT-J based word completion achieved the highest average WSCV ITR (75.36 bits/min), with GPT-2, LLaMA, Falcon approximately within 1 – 2% of it. WSCV schemes consistently produced lower variance than ASCV training. For LLaMA, the WSCV standard deviation was 7.2 bits/min, approximately one third of the ASCV value of 20.4 bits/min. Note that the width of each violin plot represents the distribution of ITR values across subjects, with wider regions indicating a higher density of subjects at that performance level. In ASCV training, similar performance trends are seen with the GPT-Neo configuration, which provided the highest ITR (60.22 bits/min), as shown in Fig 10 and Table 2. As with WSCV, the recent decoder-based LLM configurations (GPT-2, GPT-Neo, GPT-NeoX, OPT, Falcon, MPT, and LLaMA) fall within 1 – 2% of peak ASCV performance. Fig 10 shows the ASCV equivalent of Fig 9.

In Figs 9, 10 and Table 2, the character and word performance bounds computed for each subject (as described in Section) are also shown. These limits are a means of studying the efficacy of each particular scheme compared with the best achievable. The performance of the diagonal scheme is approximately within 4% of the character bound (54.43 vs. 55.86 bits/min for WSCV and 36.51 vs. 37.97 bits/min for ASCV). Following similar trends, regardless of the WSCV/ASCV training methodology, GPT-2/GPT-Neo/LLaMA/MPT/Falcon achieve performance to approximately within 5% of the word bound (for example, Falcon7 is 74.91 vs 78.21 bits/min for WSCV and 59.89 vs 62.84 bits/min for ASCV).

Table 3. Selected paired comparisons among key decoding schemes. Differences are computed within subject and reported in bits/min as the first scheme minus the second scheme. Confidence intervals were computed using paired bootstrap resampling across subjects. *p*-values are from two-sided Wilcoxon signed-rank tests with Holm–Bonferroni correction applied within each cross-validation mode. Effect size denotes the rank-biserial effect size.

Mode	Comparison	Median Δ ITR [95% CI]	Holm-adjusted *p*	Effect Size
WSCV	Diagonal – Random	2.40 [2.17, 2.67]	<0.001	1.000
WSCV	Dijkstra – Diagonal	8.99 [8.71, 9.36]	<0.001	1.000
WSCV	GPT-2 – Dijkstra	10.92 [10.38, 11.21]	<0.001	1.000
WSCV	LLaMA 3.3 – Dijkstra	10.67 [10.09, 10.97]	<0.001	1.000
WSCV	GPT-2 – LLaMA 3.3	0.40 [0.28, 0.50]	<0.001	0.883
WSCV	GPT-2 – Word Bound	–2.00 [–2.15, –1.87]	<0.001	–0.999
WSCV	LLaMA 3.3 – Word Bound	–2.46 [–2.60, –2.33]	<0.001	–0.999
WSCV	GPT-2 – GPT-J	0.18 [0.07, 0.29]	0.002	0.409
ASCV	Diagonal – Random	2.87 [2.52, 3.14]	<0.001	1.000
ASCV	Dijkstra – Diagonal	10.79 [10.43, 10.92]	<0.001	1.000
ASCV	GPT-2 – Dijkstra	15.56 [14.57, 15.90]	<0.001	1.000
ASCV	LLaMA 3.3 – Dijkstra	15.32 [14.41, 15.75]	<0.001	1.000
ASCV	GPT-2 – LLaMA 3.3	0.14 [0.10, 0.19]	<0.001	0.505
ASCV	GPT-2 – Word Bound	–3.42 [–3.64, –3.15]	<0.001	–1.000
ASCV	LLaMA 3.3 – Word Bound	–3.51 [–3.71, –3.34]	<0.001	–1.000
ASCV	GPT-2 – GPT-Neo	0.00 [–0.02, 0.03]	0.952	0.008

https://doi.org/10.1371/journal.pone.0349281.t003

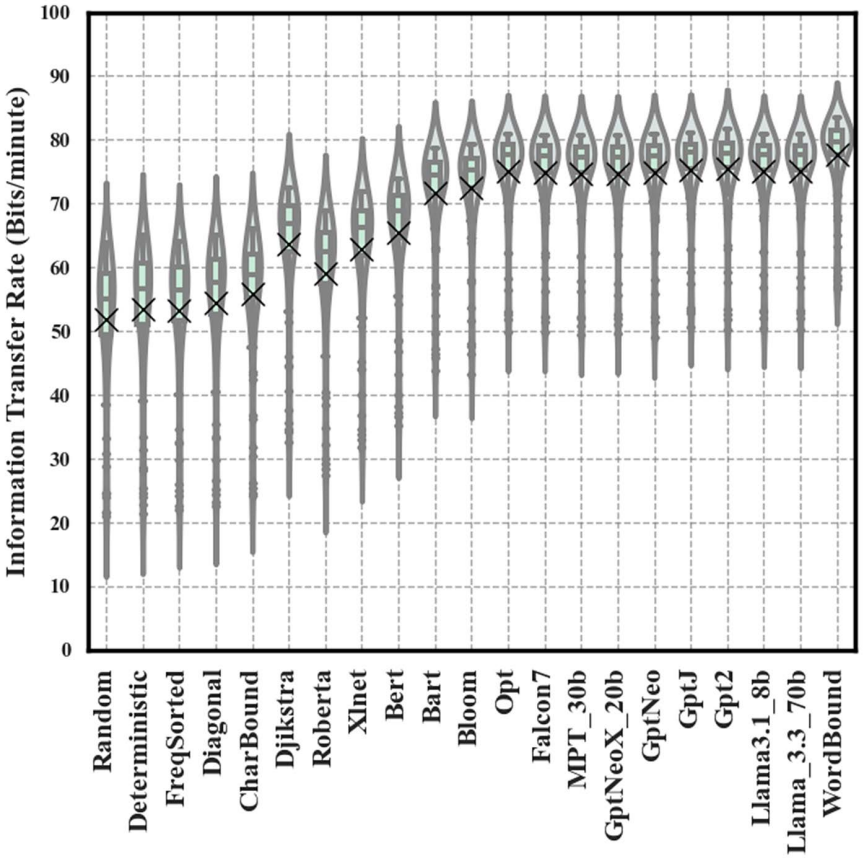


Fig 9. ITR performance comparison of different schemes for the "Within Subjects" (WSCV) training.

https://doi.org/10.1371/journal.pone.0349281.g009

Fig 11 shows the performance of traditional schemes augmented with effective word completion and prediction mechanisms. As shown in Fig 4, by employing the virtual flashboard concept, LLM-based word completion schemes can act as a simple overlay on conventional flashboards while achieving near-optimal performance with respect to the language-model-driven decoding bound.

Discussion

The central finding of this study is that the best-performing LLM-assisted P300 speller configurations approach the framework-specific word-prediction bound, suggesting that much of the gain available from language modeling alone has already been realized within the current decoding framework. In particular, high-performing models such as GPT-2, GPT-Neo, GPT-J, GPT-NeoX, OPT, Falcon, MPT, and LLaMA achieve ITR values within approximately $3-5\%$ of the theoretical word-prediction bound, whereas earlier or less well-matched architectures such as RoBERTa, BERT, XLNet, and BART show larger gaps.

In the within-subject (WSCV) analysis, the diagonal scheme achieved a mean ITR of 54.4 bits/min, modestly outperforming the random scheme (51.9 bits/min, $p < 0.001$) in all reported metrics. Importantly, as illustrated by the virtual flashboard technique shown in Figs 4 and 5, the underlying flashboard structure remains unchanged; it is the highlighting of the characters that varies with the particular scheme used. Hence, this is expected to have a negligible impact on system complexity and user attention.

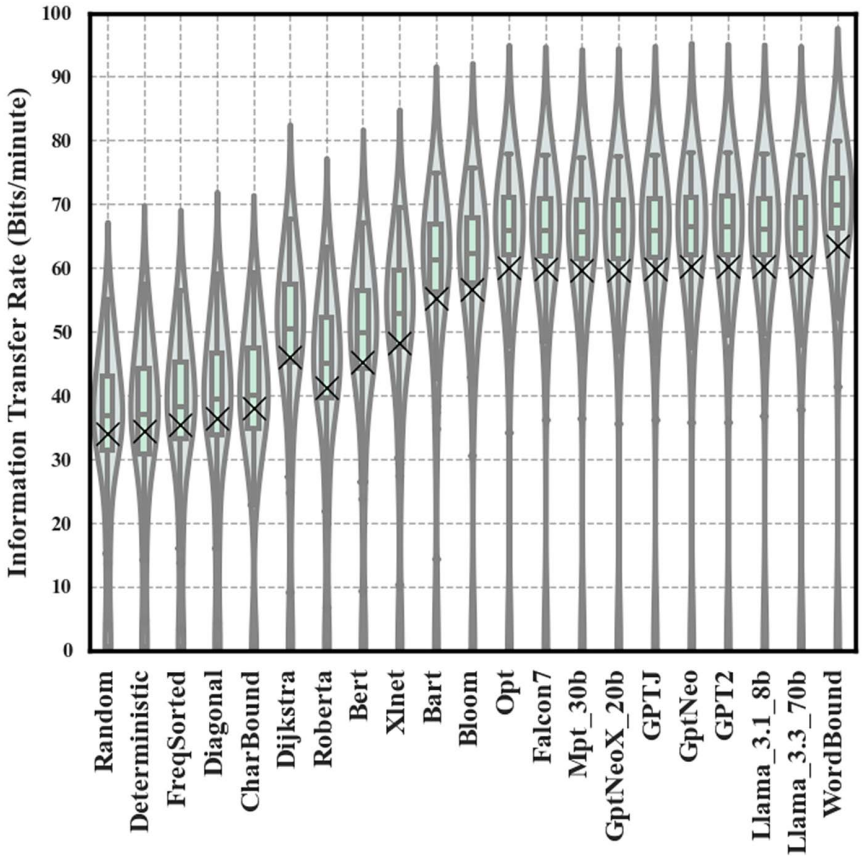


Fig 10. ITR performance comparison of different schemes for the "Across Subjects" (ASCV) training.

https://doi.org/10.1371/journal.pone.0349281.g010

In Figs 9 and 10, when the Dijkstra algorithm is used for word completion, the WSCV ITR increases to 63.6 bits/min, representing an approximately 17% improvement over the diagonal scheme and an approximately 22% improvement over the random baseline. When GPT-2 is used in conjunction with Dijkstra's algorithm, the ITR further improves to 75.3 bits/min. This corresponds to an approximately 45% improvement over the random baseline and an approximately 18% improvement over Dijkstra-based word completion alone. This value falls within 4% of the word performance limit, indicating a limited residual gain from further improvements to the language model. Because correction logs were not retained for all decoding schemes and do not support retrospective threshold sweeps, correction-burden summaries are reported only for the retained configurations in Table 1. To further increase ITR, one could potentially explore predicting not only the next word, but also performing multi-word predictions. However, regardless of the specific algorithm used, word completion algorithms provide substantial gains over schemes that rely solely on character prediction, consistent with previous work [6]. It is worth noting that by applying smoothing, flashboard character probabilities for 17 biwords in the DoI dataset that were out of vocabulary (OOV) from the biword language model were obtained by transitioning to the word and trigram models.

Across subjects (ASCV), performance (Fig 10) shows a trend similar to WSCV. However, compared to WSCV, more subjects exhibit lower ITRs as shown in Fig 10. The diagonal character flashboard showed a 7.5% improvement over the regular flashboard with random flashing, while Dijkstra's and GPT-2 word completion provide 35.7% and 76% gain, respectively. Overall, these results demonstrate that the gains generalize across both within-subject and across-subject

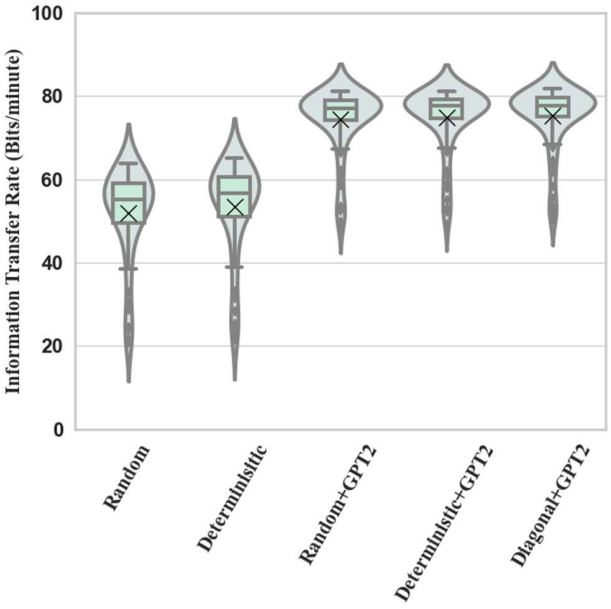


Fig 11. Violin plot of the information transfer rate of conventional flashboards, showing the effect of word completion on performance improvement for (a) "Within Subjects" (WSCV) and (b) "Across Subjects" (ASCV). NOTE: "Regular" refers to the traditional random scan with alphabetical flashboard while "Regular-Wcomp" refers to the same but with word suggestions included.

https://doi.org/10.1371/journal.pone.0349281.g011

classifiers. This is particularly important for practical deployment, as across-subject classifiers significantly reduce or eliminate the need for subject-specific calibration.

A key finding is that performance converges among the high-performing LLM-based schemes rather than being dominated by a single model architecture. In both within-subject and across-subject evaluations, recent decoder-based models such as GPT-2, GPT-Neo, GPT-J, GPT-NeoX, OPT, Falcon, MPT, and LLaMA achieved similar ITR values, typically within $1-2\%$ of the best-performing model in each training paradigm. Earlier or less well-matched architectures, including RoBERTa, BERT, XLNet, and BART, still improved performance relative to character-only baselines but showed larger gaps from the word bound. This pattern suggests that, once word suggestions are sufficiently accurate within the evaluated framework, further language-model scaling provides diminishing returns compared with improvements in neural decoding and signal acquisition.

The theoretical ceiling complements the word prediction bound introduced in Section, which captures the limits of language-model-driven decoding. Together, they allow the total system-level performance gap to be partitioned into a language modeling component and a neural decoding component.

As shown in Table 4, the word performance bound of 78.21 bits/min corresponds to approximately 76% of the conservative theoretical ceiling and roughly 38% of the idealized single-sequence ceiling. The gap between the word bound and the physiological ceiling, approximately $25-129$ bits/min, cannot be recovered through improvements in language modeling alone. This remaining gap is primarily governed by the number of flash sequences required for reliable P300 detection, as well as the fidelity of EEG signal acquisition and the efficiency of neural decoding. Improvements in classifier design and signal processing therefore represent the most promising avenues for further performance gains beyond what language modeling can provide.

These findings represent a shift in understanding compared to prior work [12]. Rather than viewing language models as a continuously improvable component driving system performance, our results indicate that the primary bottleneck

Table 4. Comparison of WSCV ITR values against the theoretical Shannon ceiling derived from McFarland et al. [41]. Percentages are relative to the conservative ceiling (∼103 bits/min), assuming a minimum of two flash sequences per selection.

Scheme	ITR (bits/min)	% of Conservative Ceiling
Random (baseline)	51.93	50%
Diagonal scheme	54.43	53%
Best LLM (GPT-J, WSCV)	75.36	73%
Word bound (WSCV)	78.21	76%
Conservative ceiling	∼103	100%
Single-sequence ceiling	∼207	—

https://doi.org/10.1371/journal.pone.0349281.t004

may now lie in other aspects of the system, such as neural signal quality, feature extraction, or decoding strategies. In this context, improvements in EEG acquisition, signal processing, and adaptive decoding may provide greater benefits than further scaling of these models.

Retrofitting, which refers to the addition of word prediction to standard speller schemes, enables performance to approach near-optimal levels in both within-subject and across-subject analyses (Fig 11). In particular, the high-performing decoder-based LLM schemes show convergent performance, suggesting that existing speller systems can be significantly enhanced through the integration of language-model-based decoding without requiring fundamental changes to their structure. More importantly, this result reinforces the conclusion that contextual word prediction plays a dominant role in achieving near-optimal performance, whereas further gains from increasingly complex models are likely to be limited once word suggestions are sufficiently accurate within the evaluated framework.

A primary limitation of this study is the reliance on offline simulation. Although offline evaluation enables systematic comparison across many language models, classifier configurations, and flashboard schemes, it does not fully capture the constraints of real-time P300 speller use. In practical online operation, performance may be affected by fatigue, attention fluctuations, feedback-driven adaptation, user frustration during repeated corrections, and latency associated with language-model inference. These factors could reduce the absolute ITR achieved in real-time use relative to the values reported here.

Accordingly, the results should be interpreted primarily as comparative performance estimates within a controlled offline framework, rather than as definitive real-time deployment benchmarks. The near-bound performance observed in this study indicates that language-model priors provide limited additional room for improvement under the evaluated assumptions, but online validation is needed to determine the extent to which these gains transfer to real users under sustained use. Future work should evaluate the proposed framework in real-time P300 spelling experiments, including measurements of fatigue, attention stability, correction burden, user adaptation, and end-to-end latency.

Furthermore, the use of simulation enables evaluation over long text passages containing rare and out-of-vocabulary (OOV) words, which are difficult to study in controlled online experiments where subjects typically generate shorter, more common phrases. As a result, the present study focuses on comparative evaluation of language model performance across system configurations, rather than establishing absolute real-time ITR benchmarks.

Another limitation concerns generalizability to the target clinical population. The dataset used in this study was collected from healthy adults aged 20–35, whereas the intended clinical users of P300 speller systems include individuals with ALS and other severe motor impairments. ALS patients may exhibit reduced P300 amplitudes, increased response latency, greater trial-to-trial variability, fatigue, and changes in attention over time. These factors could reduce absolute ITR values relative to those observed in the present offline simulations.

Nevertheless, the language-model component evaluated here operates on the decoded text stream and the candidate word set, rather than directly on disease-specific neural features. Therefore, we expect the relative benefit of improved

word prediction to remain relevant when reliable P300 responses can be obtained. However, the magnitude of the gains, the optimal decision threshold, correction burden, and real-time usability may differ in ALS populations. Validation in online experiments with ALS patients is therefore required before drawing conclusions about clinical deployment.

GPT-2 [17] has well-documented contextual limitations, and language acquisition involves more than word prediction alone. However, advances in GPT-based language models, such as GPT-3 [48] and subsequent models, mitigate these limitations, further enhancing their value. Thus, we do not view this as a fundamental limitation of the results in this paper; rather, it opens avenues for experiments with new models that are more context-sensitive.

Overall, the gap between theoretical and practical P300 speller performance has substantially narrowed, shifting the primary bottleneck from algorithmic design to physiological and hardware constraints.

Conclusion and future directions

This study investigated the role of large language models (LLMs) in improving the performance of P300 speller brain-computer interfaces, with a particular focus on understanding their framework-specific oracle-based performance bounds. By evaluating a wide range of modern LLMs within a unified framework and introducing an idealized model to establish theoretical bounds, we provide a systematic analysis of achievable performance.

Across both training paradigms, the evaluated models substantially improved typing speed, raising ITR from approximately 52 to over 75 bits/min in within-subject settings and from 34 to approximately 60 bits/min in across-subject settings. Importantly, these gains were consistent across multiple high-performing model architectures, with the best-performing LLM-based schemes operating within $3-5\%$ of the oracle-based word bound defined within the evaluated decoding framework.

The most consequential finding is not the magnitude of these gains, but their convergence. Among the high-performing LLM-based schemes, performance clustered within 5% of the theoretical word-prediction limit within this framework, indicating that further sophistication of the language model is unlikely to produce a significant additional benefit for this application. This represents a fundamental shift in where the performance bottleneck lies: the limiting factor in modern P300 spellers is no longer the quality of language modeling, but the fidelity of neural signal acquisition and decoding. Future P300 speller improvements will therefore depend more on advances in neural decoding and signal acquisition than on further language model scaling.

In addition, the use of cross-subject classifiers demonstrates the feasibility of reducing or eliminating subject-specific calibration, which is a key requirement for scalable real-world deployment. The combination of near-optimal language-model-based decoding and calibration-free classification represents a significant step toward practical BCI communication systems.

Future work will focus on validating these findings in online experiments and exploring complementary approaches, such as adaptive thresholding and improved neural decoding techniques, to further enhance system performance.

Supporting information

S1 Appendix. Kneser–Ney smoothing formulation.
(PDF)

Author contributions

Data curation: DongWoo Lee.

Investigation: Nithin Parthasarathy, Nitya Parthasarathy, William Speier.

Methodology: William Speier.

Software: Nithin Parthasarathy.

Supervision: Corey Arnold, Nader Pouratian, William Speier.

Validation: James Soetedjo.

Writing – original draft: Nithin Parthasarathy.

Writing – review & editing: Nithin Parthasarathy, Saarang Panchavati, William Speier.

References

1. van Es MA, Hardiman O, Chio A, Al-Chalabi A, Pasterkamp RJ, Veldink JH, et al. Amyotrophic lateral sclerosis. Lancet. 2017;390(10107):2084–98. https://doi.org/10.1016/S0140-6736(17)31287-4 PMID: 28552366
2. Gao S, Wang Y, Gao X, Hong B. Visual and auditory brain-computer interfaces. IEEE Trans Biomed Eng. 2014;61(5):1436–47. https://doi.org/10.1109/TBME.2014.2300164 PMID: 24759277
3. McFarland DJ, Sarnacki WA, Townsend G, Vaughan T, Wolpaw JR. The P300-based brain-computer interface (BCI): effects of stimulus rate. Clin Neurophysiol. 2011;122(4):731–7. https://doi.org/10.1016/j.clinph.2010.10.029 PMID: 21067970
4. Farwell LA, Donchin E. Talking off the top of your head: toward a mental prosthesis utilizing event-related brain potentials. Electroencephalogr Clin Neurophysiol. 1988;70(6):510–23. https://doi.org/10.1016/0013-4694(88)90149-6 PMID: 2461285
5. Townsend G, LaPallo BK, Boulay CB, Krusienski DJ, Frye GE, Hauser CK, et al. A novel P300-based brain-computer interface stimulus presentation paradigm: moving beyond rows and columns. Clin Neurophysiol. 2010;121(7):1109–20. https://doi.org/10.1016/j.clinph.2010.01.030 PMID: 20347387
6. Speier W, Arnold C, Chandravadia N, Roberts D, Pendekanti S, Pouratian N. Improving P300 Spelling Rate using Language Models and Predictive Spelling. Brain Comput Interfaces (Abingdon). 2018;5(1):13–22. https://doi.org/10.1080/2326263X.2017.1410418 PMID: 30560145
7. Speier W, Arnold C, Pouratian N. Integrating language models into classifiers for BCI communication: a review. J Neural Eng. 2016;13(3):031002. https://doi.org/10.1088/1741-2560/13/3/031002 PMID: 27153565
8. Speier W, Arnold CW, Deshpande A, Knall J, Pouratian N. Incorporating advanced language models into the P300 speller using particle filtering. J Neural Eng. 2014.
9. Kindermans PJ, Verschore H, Verstraeten D, Schrauwen B. A P300 BCI for the masses: Prior information enables instant unsupervised spelling. In: Advances in Neural Information Processing Systems. vol. 25. 2012. p. 719–27.
10. Kindermans P-J, Tangermann M, Müller K-R, Schrauwen B. Integrating dynamic stopping, transfer learning and language models in an adaptive zero-training ERP speller. J Neural Eng. 2014;11(3):035005. https://doi.org/10.1088/1741-2560/11/3/035005 PMID: 24834896
11. Parthasarathy N, Arnold C, Pouratian N, Speier W. Optimizing P300 speller performance using language models for character and word prediction. In: IEEE Bioinformatics and Bioengineering Conference. 2020.
12. Parthasarathy N, Soetedjo J, Panchavati S, Parthasarathy N, Arnold C, Pouratian N, et al. High performance P300 spellers using GPT2 word prediction with cross-subject training. Brain-Comput Interfaces. 2024;11(4):210–24. https://doi.org/10.1080/2326263x.2024.2413214
13. Lebedev MA, Makarova A, Kleeva D, Maysuradze AI. Connecting a P300 speller to a large language model. BioRxiv. 2025. https://doi.org/10.1101/2025.11.06.686984
14. Hong J, Wang W, Najafizadeh L. ChatBCI, a P300 speller BCI with context-driven word prediction leveraging large language models, from concept to evaluation. Sci Rep. 2026;16(1):6379. https://doi.org/10.1038/s41598-025-25660-7 PMID: 41698950
15. Liu Y, Ott M, Goyal N, Du J, Joshi M, Chen D, et al. RoBERTa: A Robustly Optimized BERT Pretraining Approach. ArXiv:1907.11692.
16. Devlin J, Chang M, Lee K, Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding;. ArXiv:1810.04805.
17. Radford A, Wu J, Child R, Luan D, Amodei D, Sutskever I. Language models are unsupervised multitask learners. OpenAI; 2019.
18. LLaMA 3.1: Open-Source Large Language Model. 2024. Available from: https://ai.meta.com/blog/meta-llama-3-1/
19. LLaMA 3.3: Multilingual Large Language Model. 2024. Available from: https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct
20. Almazrouei E, Alobeidli H, Alshamsi A, Cappelli A, Cojocaru R, Debbah M, et al. The Falcon Series of Open Language Models. 2023. ArXiv:2311.16867 p. Available from: https://arxiv.org/abs/2311.16867
21. Lewis M, Liu Y, Goyal N, Ghazvininejad M, Abdelrahman M, Levy O, et al. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. ArXiv:1910.13461.
22. Yang Z, Dai Z, Yang Y, Carbonell J, Salakhutdinov R, Le Q. XLNet: Generalized Autoregressive Pretraining for Language Understanding. ArXiv:1906.08237.
23. Black S, Gao L, Wang P, Connor L, Biderman S. GPT-Neo: Large Scale Autoregressive Language Modeling with Mesh-Tensorflow. 2021. Available from: https://doi.org/10.5281/zenodo.5297715
24. Scao TL, Fan A, Akiki C, et al. BLOOM: A 176B-Parameter Open-Access Multilingual Language Model. 2022. ArXiv:2211.05100 p. Available from: https://doi.org/10.48550/arXiv.2211.05100
25. Zhang H, Niu X, et al. OPT: Open Pretrained Transformer Language Models. Available from: https://doi.org/10.48550/arXiv.2205.01068

26. Wang B, Komatsuzaki A. GPT-J-6B: A 6 Billion Parameter Autoregressive Language Model. EleutherAI; 2021. Available from: https://doi.org/10.48550/arXiv.2104.06807
27. Black S, Mccarthy J. GPT-NeoX: Large-scale autoregressive language model. Available from: https://doi.org/10.48550/arXiv.2204.09216
28. MosaicML NLP Team. Introducing MPT-30B: Raising the bar for open-source foundation models. MosaicML Technical Blog. 2023. Available from: https://www.mosaicml.com/blog/mpt-30b
29. Lewand RE. Cryptological Mathematics. Washington, DC: Mathematical Association of America; 2000.
30. Jurafsky D, Martin J. Speech and language processing: an introduction to natural language processing, computational linguistics, and speech recognition. Prentice Hall; 2009.
31. Kindermans P-J, Verschore H, Schrauwen B. A unified probabilistic approach to improve spelling in an event-related potential-based brain-computer interface. IEEE Trans Biomed Eng. 2013;60(10):2696–705. https://doi.org/10.1109/TBME.2013.2262524 PMID: 23674419
32. Francis W, Kucera H. Brown Corpus Manual. 1979.
33. Lu J, Speier W, Hu X, Pouratian N. The effects of stimulus timing features on P300 speller performance. Clin Neurophysiol. 2013;124(2):306–14. https://doi.org/10.1016/j.clinph.2012.08.002 PMID: 22939456
34. Sellers EW, Krusienski DJ, McFarland DJ, Vaughan TM, Wolpaw JR. A P300 event-related potential brain-computer interface (BCI): the effects of matrix size and inter stimulus interval on performance. Biol Psychol. 2006;73(3):242–52. https://doi.org/10.1016/j.biopsycho.2006.04.007 PMID: 16860920
35. Draper NR, Smith H. Applied Regression Analysis. New York (NY): Wiley; 1981.
36. Cormen TH, Leiserson CE, Rivest RL, Stein C. Introduction to Algorithms. 3rd ed. MIT Press; 2009.
37. Dijkstra EW. A note on two problems in connexion with graphs. Numer Math. 1959;1(1):269–71. https://doi.org/10.1007/bf01386390
38. Radford A, Narasimhan K, Salimans T, Sutskever I. Improving Language Understanding by Generative Pre-Training. OpenAI; 2018. Available from: https://openai.com/research/language-unsupervised
39. Speier W, Deshpande A, Pouratian N. A method for optimizing EEG electrode number and configuration for signal acquisition in P300 speller systems. Clin Neurophysiol. 2015;126(6):1171–7. https://doi.org/10.1016/j.clinph.2014.09.021 PMID: 25316166
40. Declaration of Independence. Available from: http://ushistory.org/declaration/document
41. McFarland DJ, Sarnacki WA, Wolpaw JR. Brain-computer interface (BCI) operation: optimizing information transfer rates. Biol Psychol. 2003;63(3):237–51. https://doi.org/10.1016/s0301-0511(03)00073-5 PMID: 12853169
42. Pierce J. An introduction to information theory. New York: Dover; 1980.
43. Gogna Y, Tiwari S, Singla R. Cognitive Load with Different Origins: An EEG Model-Based Explanation. NeuroQuantology. 2022;20(14):743–8.
44. Kshirsagar GB, Londhe ND. Improving Performance of Devanagari Script Input-Based P300 Speller Using Deep Learning. IEEE Trans Biomed Eng. 2019;66(11):2992–3005. https://doi.org/10.1109/TBME.2018.2875024 PMID: 30307853
45. Speier W, Arnold C, Lu J, Deshpande A, Pouratian N. Integrating language information with a hidden Markov model to improve communication rate in the P300 speller. IEEE Trans Neural Syst Rehabil Eng. 2014;22(3):678–84. https://doi.org/10.1109/TNSRE.2014.2300091 PMID: 24760927
46. Corder G, Foreman D. Nonparametric Statistics: A Step-by-Step Approach. Wiley; 2014.
47. Wilcoxon F. Individual Comparisons by Ranking Methods. Biom Bull. 1945;1(6):80. https://doi.org/10.2307/3001968
48. Brown T, Mann B, Ryder N, et al. Language Models are Few-Shot Learners. Adv Neural Inform Process Syst. 2020;33:ArXiv:2005.14165.